

## VAE、Diffusion、Flow Matching 系统讲解（二） Diffusion 扩散模型完全指南



冰锐

吉大车辆工程

关注他

1 人赞同了该文章

### （二）Diffusion 扩散模型完全指南：从前向加噪到 Score Matching

**【系列文章导航】** VAE、Diffusion 与 Flow Matching 系统性深度讲解

- （一）VAE 变分自编码器
- （二）Diffusion 扩散模型完全指南（本篇）
- （三）Flow Matching 流匹配
- （四）三者对比、Stable Diffusion/DiT 与进阶话题

在上一篇中我们系统讲解了 VAE 的变分下界、重参数化与编码-解码结构；它与 Diffusion 共享同一套「引入隐变量 + 最大化似然/ELBO」的概率图式思路。本篇进入 Diffusion：先由固定的前向过程把数据渐变成噪声、再用可学的反向过程逐步去噪，并沿「为何  $q(x_{t-1}|x_t)$  不可算  $\rightarrow$  条件后验可配方  $\rightarrow$  预测噪声」与「ELBO 分解为逐步 KL  $\rightarrow$  简化为噪声 MSE」两条主线，把前向/反向与训练目标一次讲透。

## 2. Diffusion Models<sup>+</sup>（扩散模型）

### 2.1 核心思想

如果我知道如何把一张图片逐渐“腐蚀”成噪声，那么学会“逆转腐蚀”的过程就等于学会了生成。

两步走：

- 前向过程**（固定，不需要学习）：对数据逐步加高斯噪声， $T$  步后变成纯噪声
- 反向过程**（需要学习）：训练神经网络逐步去噪，把纯噪声变回数据

前向过程（加噪，固定）：

$x_0 \quad x_1 \quad x_2 \quad x_T$   
[清晰图片]  $\rightarrow$  [微噪声]  $\rightarrow$  [更多噪声]  $\rightarrow \dots \rightarrow$  [纯高斯噪声]  
 $\longrightarrow$  每步加一点噪声  $\epsilon_t$ ，信号逐渐衰减  $\longrightarrow$

反向过程（去噪，学习）：

$x_T \quad x_{\{T-1\}} \quad x_1 \quad x_0$   
[纯噪声]  $\rightarrow$  [稍微去噪]  $\rightarrow \dots \rightarrow$  [几乎清晰]  $\rightarrow$  [清晰图片]  
 $\longleftarrow$  网络预测  $\epsilon_\theta(x_t, t)$ ，逐步恢复信号  $\longleftarrow$

信噪比  $SNR(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$ ：

$t=0$  高SNR（几乎纯信号）  
 $t=T/4$    
 $t=T/2$    
 $t=3T/4$    
 $t=T$  低SNR（几乎纯噪声）

已赞同 1



添加评论



收藏



喜欢



分享



申请转载



#### 关于作者



冰锐

吉大车辆工程

回答

7

文章

48

关注者

3,597

关注他

发私信

#### 大家都在搜

换一换

- 美国白宫记者晚宴发生枪… 367 万 新
- 男子性情大变确诊神经梅… 366 万 热
- DeepSeek V4 预览版上线… 333 万
- AC 米兰国米球员被曝集体… 310 万
- 华谊兄弟被正式申请破产 297 万
- 爱喝无糖饮料的天塌了 297 万
- 东方甄选4名主播集体离职 283 万 新
- OpenAI 发布 GPT-5.5 270 万
- 男童疑被误诊为脑瘫治疗 7… 258 万
- 张军 253 万



## 2.7 Score Matching 视角: 预测噪声 $\approx$ 学概率密度的梯度场 $\rightarrow$ 连接到 Flow Matching

立高斯噪声的线性组合。

由独立高斯的可加性：如果  $a\epsilonpsilon_1 + b\epsilonpsilon_2$ ，其中  $\epsilonpsilon_1, \epsilonpsilon_2 \sim \mathcal{N}(0, I)$  独立，则  $a\epsilonpsilon_1 + b\epsilonpsilon_2 \sim \mathcal{N}(0, (a^2 + b^2)I)$ 。

所以方差为  $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1\alpha_2 = 1 - \bar{\alpha}_2$

因此： $x_2 = \sqrt{\bar{\alpha}_2}x_0 + \sqrt{1 - \bar{\alpha}_2}\epsilonpsilon$ ， $\epsilonpsilon \sim \mathcal{N}(0, I)$

用数学归纳法推广到任意  $t$ ：

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

等价写法：

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilonpsilon, \quad \epsilonpsilon \sim \mathcal{N}(0, I)$$

当  $t = T$  时， $\bar{\alpha}_T = \prod_{s=1}^T (1 - \beta_s) \approx 0$ （因为每一步都在乘一个小于 1 的数），所以  $x_T \approx \mathcal{N}(0, I)$ 。

### 2.2.3 信噪比的演变

$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilonpsilon$  可以看作信号和噪声的混合：

- **信号部分**： $\sqrt{\bar{\alpha}_t}x_0$ ，强度为  $\bar{\alpha}_t$
- **噪声部分**： $\sqrt{1 - \bar{\alpha}_t}\epsilonpsilon$ ，强度为  $1 - \bar{\alpha}_t$
- **信噪比 (SNR)**： $\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$

随着  $t$  增大，SNR 从大（接近纯信号）单调递减到接近 0（接近纯噪声）。

## 2.3 反向过程（去噪）——Diffusion 的核心难题与解法

**本节是理解 Diffusion 的关键。**前向加噪很简单（上节已完成），但反向去噪是一个深刻的数学问题。本节的逻辑链：

1. 我们想要什么？→ 知道  $q(x_{t-1}|x_t)$  就能从噪声生成数据
2. 为什么得不到？→ 需要知道整个数据分布（正是我们想学的东西）
3. 怎么绕过？→ 给定  $x_0$  后，后验变成三个已知高斯的贝叶斯运算
4. 可算的后验长什么样？→ [高斯分布<sup>+</sup>](#)，均值和方差有闭式解（详细配方推导）
5. 推理时没有  $x_0$  怎么办？→ 用  $\epsilonpsilon$  替换  $x_0$  → 训练神经网络预测  $\epsilonpsilon$

### 2.3.1 目标：我们需要什么？

前向过程把数据  $x_0$  逐步变成噪声  $x_T \sim \mathcal{N}(0, I)$ 。**生成数据就是反过来**：从噪声  $x_T$  出发，一步步去噪，最终得到  $x_0$ 。

要做到这一点，我们需要知道**反向转移概率**  $q(x_{t-1}|x_t)$ ：给定当前的噪声图  $x_t$ ，上一步的  $x_{t-1}$  应该长什么样？

如果我们知道所有时间步的  $q(x_{t-1}|x_t)$ ，就可以从  $x_T$  开始逐步采样  $x_{T-1}, x_{T-2}, \dots, x_0$ ，完成生成。

$$q(x_{-t}|x_t) = \frac{q(x_t|x_{-t-1}) \cdot q(x_{-t-1})}{q(x_t)}$$

- $q(x_t|x_{-t-1})$ : 前向加噪的定义，是已知的高斯 → **没问题**
  - $q(x_{-t-1})$  和  $q(x_t)$ : **边际分布** (marginal distribution) → **问题在这里!**

**$q(x_{-t-1})$  是什么?** 它是”把所有可能的原始数据  $x_0$  各自加噪到第  $t-1$  步后，混合在一起” 的分布:

$$q(x_{-t-1}) = \int q(x_{-t-1}|x_0) \cdot q_{\text{data}}(x_0) \, dx_0$$

每个  $x_0$  对应一个高斯  $q(x_{-t-1}|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{-t-1}}x_0, (1-\bar{\alpha}_{-t-1})I)$ , 但  $q_{\text{data}}(x_0)$  是真实数据分布——**就是我们不知道的东西** (如果知道了, 就不需要训练生成模型了! )。所以这个积分没法算。

**具体例子:** 假设数据集有 100 万张图片。  $q(x_{-t-1})$  就是 100 万个高斯的加权混合:

$$q(x_{-t-1}) \approx \frac{1}{N} \sum_{i=1}^N \mathcal{N}(x_{-t-1}; \sqrt{\bar{\alpha}_{-t-1}}x_0^{(i)}, (1-\bar{\alpha}_{-t-1})I)$$

这是一个极其复杂的多峰分布，不是简单的高斯，也没有闭式表达。

**注意区分: 高斯的”线性组合”与”混合”是两个完全不同的运算。** 两者公式看似相似, 但核心区别在于  $\mathcal{N}$  的角色和加权的对象。

**线性组合**  $Z = aX + bY$  ( $X, Y$  为随机变量):  $\mathcal{N}$  在”幕后”描述  $X, Y$  的取值规律, 公式本身是对随机变量的**采样值** (数字) 做算术运算。采样时  $X$  和  $Y$  **同时取值**,  $Z$  是两个数值的加权和。结果仍是高斯。

**混合**  $p(x) = \sum_i w_i \cdot \mathcal{N}(x; \mu_i, \sigma_i^2)$ :  $\mathcal{N}$  **直接出现在等号右边**, 作为函数被加权。公式是在对**概率密度函数**做加权平均, 定义一个新的密度函数。采样时先按权重  $w_i$  选中一个分量, 再**仅从该分量**采样, 其余分量不参与。结果一般不是高斯。

|       | 线性组合 $aX+bY$      | 混合 $\sum_i w_i \cdot \mathcal{N}_i(x)$ |
|-------|-------------------|----------------------------------------|
| N 的角色 | 幕后描述 $X, Y$ 的分布   | 直接出现在等号右边, 被加权                         |
| 加权对象  | 随机变量的采样值 (数字)     | 概率密度函数 (函数)                            |
| 采样方式  | 同时从所有分布各取一个值, 做算术 | 先选一个分布, 只从它采样                          |
| 结果    | 高斯                | 一般不是高斯 (多峰)                            |

这里的  $q(x_{-t-1})$  属于后者——每张图片  $x_0^{(i)}$  贡献一个高斯分量, 它们以等概率混合。而前向过程中  $\sqrt{\alpha_2(1-\alpha_1)}\epsilon_1 + \sqrt{1-\alpha_2}\epsilon_2$  属于前者——两个噪声同时取值再做加法, 结果仍是高斯。

**小结:**  $q(x_{-t-1}|x_t)$  不可算的根本原因——**分母中的边际分布要对整个数据分布积分, 而数据分布正是我们想要学习的东西。** 这是一个逻辑上的”循环依赖”。

### 2.3.3 关键转折: 给定 $x_0$ 后, 一切变成已知高斯

现在换一个问题: 如果我们**同时知道**  $x_t$  和原始数据  $x_0$ , 能不能算出  $x_{-t-1}$  的分布?

用贝叶斯定理:

$$q(x_{-t-1}|x_t, x_0) = \frac{q(x_t|x_{-t-1}, x_0) \cdot q(x_{-t-1}|x_0)}{q(x_t|x_0)}$$

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

直答



消息

私信

|                    |                 |                                                            |
|--------------------|-----------------|------------------------------------------------------------|
| $q(x_{t-1}   x_t)$ | 节)              | $q(x_t   x_{t-1})$                                         |
| $q(x_{t-1}   x_0)$ | 前向闭式解 (2.2.2 节) | $N(\sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})I)$ |
| $q(x_t   x_0)$     | 前向闭式解 (2.2.2 节) | $N(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$         |

注： $q(x_t | x_{t-1}, x_0) = q(x_t | x_{t-1})$ ，因为前向过程是**马尔可夫**的——下一步只依赖当前步，与更早的步无关。

### 为什么给定 $x_0$ 后就不需要边际分布了？

对比 2.3.2 节的困境：

- **没有  $x_0$**  时：需要  $q(x_{t-1}) = \int q(x_{t-1}|x_0)q_{\text{data}}(x_0)dx_0 \rightarrow 100$  万个高斯的混合  $\rightarrow$  **不可行**
- **给定  $x_0$**  后：只需要  $q(x_{t-1}|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})I) \rightarrow$  **一个确定的高斯**  $\rightarrow$  闭式已知

**本质：**给定  $x_0$  后，“100 万个高斯的混合”坍缩成了“一个确定的高斯”。不确定性消失了，因为我们知道了数据从哪里来。

三个已知的高斯做贝叶斯运算（乘法 / 除法），结果仍然是高斯。接下来我们通过“配方”（completing the square）精确地算出这个高斯的均值和方差。

### 2.3.4 完整推导：对 $x_{t-1}$ 配方

**目标：**把  $q(x_{t-1}|x_t, x_0)$  写成  $\mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$  的形式，求出  $\tilde{\mu}_t$  和  $\tilde{\beta}_t$ 。

#### 第一步：写出对数概率（只保留含 $x_{t-1}$ 的项）

由贝叶斯定理：

$$\log q(x_{t-1}|x_t, x_0) = \log q(x_t|x_{t-1}) + \log q(x_{t-1}|x_0) - \underbrace{\log q(x_t|x_0)}_{\text{不含 } x_{t-1} \text{ 的项, 是常数}} + C$$

展开前两项的指数部分（只关注  $x_{t-1}$  的项）：

来自  $q(x_t|x_{t-1})$ ：

**为什么  $\log q(x_t|x_{t-1})$  的核心部分是  $-\frac{1}{2\beta_t}\|x_t - \sqrt{\alpha_t}x_{t-1}\|^2$ ？**

$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, \beta_t I)$ ，代入高斯密度函数  $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\{-\frac{1}{2\sigma^2}(x-\mu)^2\}$ ，取对数后归一化常数  $\log \frac{1}{\sqrt{2\pi\beta_t}}$  不含  $x_{t-1}$ （吸收进 C），只剩指数上的二次式。

$$-\frac{1}{2\beta_t}\|x_t - \sqrt{\alpha_t}x_{t-1}\|^2 = -\frac{1}{2\beta_t}(\alpha_t x_{t-1}^2 - 2\sqrt{\alpha_t}x_t x_{t-1} + x_t^2)$$

右侧展开用的是完全平方公式  $\|a - b\|^2 = a^2 - 2ab + b^2$ ，其中  $a = x_t$ ，

$b = \sqrt{\alpha_t}x_{t-1}$ ， $b^2 = (\sqrt{\alpha_t})^2 x_{t-1}^2 = \alpha_t x_{t-1}^2$ 。

来自  $q(x_{t-1}|x_0)$ （由前向闭式解代入  $t-1$ ：均值  $\mu = \sqrt{\bar{\alpha}_{t-1}}x_0$ ，方差  $\sigma^2 = 1 - \bar{\alpha}_{t-1}$ ）：

$$-\frac{1}{2(1-\bar{\alpha}_{t-1})}\|x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0\|^2 = -\frac{1}{2(1-\bar{\alpha}_{t-1})}\left(x_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}x_0 x_{t-1} + \bar{\alpha}_{t-1}x_0^2\right)$$

已赞同 1



添加评论



收藏



喜欢



分享



申请转载



...

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息

私信

高斯分布的均值和方差。高斯分布的概率密度函数为：

$$-\frac{x^2}{2\sigma^2} + \frac{\mu x}{\sigma^2} + C$$
。所以只要找出  $x_{t-1}$  和  $x_t$  的系数，就能反推出  $\sigma^2$ （方差）和  $\mu$ （均值）。

$x_{t-1}^2$  的系数（来自两项之和）：

$$-\frac{\alpha_t^2 \beta_t}{\beta_t} - \frac{1}{2(1-\bar{\alpha}_{t-1})} = -\frac{1}{2} \left( \frac{\alpha_t^2}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} \right)$$

通分：

$$\frac{\alpha_t^2}{\beta_t} + \frac{1}{\beta_t(1-\bar{\alpha}_{t-1})} = \frac{\alpha_t^2(1-\bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}$$

分子展开，利用  $\beta_t = 1 - \alpha_t$  和  $\bar{\alpha}_t = \alpha_t \cdot \bar{\alpha}_{t-1}$ ：

$$\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t = \alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \bar{\alpha}_{t-1}$$

所以  $x_{t-1}^2$  的系数为  $-\frac{1}{2} \cdot \frac{1}{1-\bar{\alpha}_{t-1}}$ 。

对于高斯  $\mathcal{N}(\mu, \sigma^2)$ ， $x^2$  的系数是  $-\frac{1}{2\sigma^2}$ ，因此：

$$\tilde{\beta}_t = \frac{\beta_t(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_{t-1}}$$

$x_{t-1}$  的一次项系数：

$$\frac{\sqrt{\alpha_t} x_t}{\beta_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} x_0}{1-\bar{\alpha}_{t-1}}$$

对于高斯  $\mathcal{N}(\mu, \sigma^2)$ ， $x$  的一次项系数是  $\frac{\mu}{\sigma^2}$ ，所以：

$$\frac{\tilde{\mu}_t}{\tilde{\beta}_t} = \frac{\sqrt{\alpha_t} x_t}{\beta_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} x_0}{1-\bar{\alpha}_{t-1}}$$

两边乘以  $\tilde{\beta}_t$ ：

$$\tilde{\mu}_t = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1-\bar{\alpha}_{t-1}} x_0$$

结论：

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t)$$

我们精确地知道了反向后验的形状——它是一个高斯，均值是  $x_t$  和  $x_0$  的加权平均，方差由噪声调度完全确定（不依赖于数据）。

**直觉理解：**  $\tilde{\mu}_t$  是  $x_t$ （当前噪声图）和  $x_0$ （原始数据）之间的“折中”。在噪声大的时候（ $t$  大），更依赖  $x_0$ ；在噪声小的时候（ $t$  小），更依赖  $x_t$ 。

### 2.3.5 把 $x_0$ 用噪声 $\epsilon$ 替换——连接到神经网络

上面的后验均值  $\tilde{\mu}_t$  依赖于  $x_0$ 。但推理时我们没有  $x_0$ ——如果有  $x_0$ ，就不需要生成模型了！

**关键一步：** 利用前向闭式解（2.2.2 节）反解  $x_0$ ：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon \quad \Rightarrow \quad x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

已赞同 1



添加评论



收藏



喜欢



分享



申请转载



...

$$\frac{\sqrt{\bar{\alpha}_{t-1}}\sqrt{\beta_t}\{1-\bar{\alpha}_t\}}{\epsilon\sqrt{\bar{\alpha}_t}}\left(x_t-\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\right)$$

注意  $\frac{\sqrt{\bar{\alpha}_{t-1}}\sqrt{\beta_t}}{\sqrt{\alpha_t}} = \frac{1}{\sqrt{\alpha_t}}$ （因为  $\bar{\alpha}_t = \alpha_t \cdot \bar{\alpha}_{t-1}$ ，所以  $\sqrt{\bar{\alpha}_t} = \sqrt{\alpha_t} \cdot \sqrt{\bar{\alpha}_{t-1}}$ ，约掉  $\sqrt{\bar{\alpha}_{t-1}}$  即可）。

**合并  $x_t$  系数的详细过程：**代入后  $x_t$  前有两个系数，分别记为 A、B：

$$A = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}, \quad B = \frac{\beta_t}{\bar{\alpha}_t\sqrt{\alpha_t}}$$

通分（公分母  $(1-\bar{\alpha}_t)\sqrt{\alpha_t}$ ），将 A 的分子分母同乘  $\sqrt{\alpha_t}$ ：

$$A + B = \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}}$$

分子展开： $\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t = \alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \bar{\alpha}_t$ （用到  $\alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$  和  $\beta_t = 1 - \alpha_t$ ）。

分子  $(1-\bar{\alpha}_t)$  与分母中的  $(1-\bar{\alpha}_t)$  约掉，得  $A+B = \frac{1}{\sqrt{\alpha_t}}$ 。

合并后：

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\right)\epsilon$$

**这个公式的意义极其深远：**

- $\tilde{\mu}_t$  只依赖于  $x_t$ （当前噪声图）和  $\epsilon$ （前向过程中加入的噪声）
- $x_t$  在推理时是已知的（就是当前状态）
- $\epsilon$  是唯一未知的量！

所以，只要训练一个神经网络  $\epsilon_{\theta}(x_t, t)$  来预测加入的噪声  $\epsilon$ ，就能用上面的公式算出  $\tilde{\mu}_t$ ，从而完成反向去噪。这就是 Diffusion 模型“预测噪声”的根本原因——不是随意的设计选择，而是数学推导的必然结果。

### 2.3.6 反向过程的完整图景

让我们回顾一下整个反向过程的逻辑链：

| 步骤                               | 内容                                           | 状态                |
|----------------------------------|----------------------------------------------|-------------------|
| 1. 想要 $q(x_{t-1}   x_t)$         | 从噪声反推数据                                      | 不可算（需要整个数据分布）     |
| 2. 退而求其次 $q(x_{t-1}   x_t, x_0)$ | 给定原始数据后的后验                                   | 可算（三个已知高斯配方）      |
| 3. 后验均值 $\mu_t(x_t, x_0)$        | 依赖 $x_t$ 和 $x_0$                             | 推理时没有 $x_0$       |
| 4. 用 $\epsilon$ 替换 $x_0$         | $\mu_t(x_t, \epsilon)$                       | 推理时不知道 $\epsilon$ |
| 5. 训练网络预测 $\epsilon$             | $\epsilon_{\theta}(x_t, t) \approx \epsilon$ | 这就是我们要训练的！        |

训练好  $\epsilon_{\theta}$  后，每一步反向去噪为：（重参数化）

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\right)\epsilon_{\theta}(x_t, t) + \sqrt{\tilde{\beta}_t}z, \quad z \sim \mathcal{N}(0, I)$$



用  $\epsilon$  算出真实的  $\mu_t$ 

不需要实际采样（只需计算损失）

用  $\epsilon_\theta$  算出近似的  $\mu_t$ 逐步采样  $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$ 

## 2.4 损失函数的推导——从变分下界到简化目标

**上一节的结论：**我们需要训练  $\epsilon_{\theta}(x_t, t)$  来预测噪声。但“预测噪声的 MSE”这个损失函数是从哪来的？是随意定义的吗？**不是**——它是从严格的变分下界（ELBO）一步步推导出来的。本节展示这个推导过程，你会看到最终的简洁形式是如何从复杂的理论框架中“自然涌现”的。

### 2.4.1 变分下界（VLB）

**目标：**我们想最大化数据的对数似然  $\log p_{\theta}(x_0)$ ，即让模型生成真实数据的概率尽可能高。但  $p_{\theta}(x_0)$  需要对所有可能的噪声链  $x_1, x_2, \dots, x_T$  积分，不可直接计算。

**思路与 VAE 完全一致**（参见 1.3 节）：引入一个已知的分布作为“桥梁”，用 Jensen 不等式推出一个可计算的下界。

| 对比   | VAE (1.3 节)                                           | Diffusion                                                                    |
|------|-------------------------------------------------------|------------------------------------------------------------------------------|
| 隐变量  | $z$ (单个向量)                                            | $x_1, x_2, \dots, x_T$ (整条马尔可夫链)                                             |
| 近似后验 | $q_{\phi}(z   x_0)$ (需要学习的 Encoder)                   | $q(x_{1:T}   x_0)$ (固定的前向加噪过程，无需学习)                                          |
| 生成模型 | $p_{\theta}(x_0, z) = p_{\theta}(x_0   z) \cdot p(z)$ | $p_{\theta}(x_{0:T}) = p(x_T) \cdot \prod_{t=1}^T p_{\theta}(x_{t-1}   x_t)$ |

**推导过程**（与 VAE 的 1.3.2-1.3.3 节完全类比）：

$$\begin{aligned} \log p_{\theta}(x_0) &= \log \int p_{\theta}(x_{0:T}) dx_{1:T} \quad \& \text{ (对隐变量积分)} \\ &= \log \int \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \cdot q(x_{1:T} | x_0) dx_{1:T} \\ &\quad \& \text{ (乘以 } q/q = 1 \text{)} \\ &= \log \mathbb{E}_{q(x_{1:T} | x_0)} \left[ \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \right] \quad \& \text{ (写成期望)} \\ &\geq \mathbb{E}_q \left[ \log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \right] \quad \& \text{ (Jensen 不等式, } \log \text{ 是凹函数)} \end{aligned}$$

最后一行就是 **ELBO**。最大化 ELBO 等价于“把  $\log p_{\theta}(x_0)$  的下界尽可能推高”。

**分解为  $T+1$  项：**

#### 详细推导过程

##### 第一步：将分子分母写成马尔可夫链的乘积

- 分子（反向生成）： $p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$
- 分母（前向加噪）： $q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$

取  $\log$  后乘积变求和：

$$\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} = \log p(x_T) + \sum_{t=1}^T \log p_{\theta}(x_{t-1} | x_t) - \sum_{t=1}^T \log q(x_t | x_{t-1})$$

##### 第二步：翻转 $q$ 的条件方向

分子中  $p_{\theta}(x_{t-1} | x_t)$  是从  $t$  到  $t-1$ ，分母中  $q(x_t | x_{t-1})$  是从  $t-1$  到  $t$ ，方向不一致。对  $t \geq 2$ ，用贝叶斯定理（在给定  $x_0$  下）翻转：



## 第二步：伸缩抵消 (telescoping)

将上式对  $t=2$  到  $T$  求和时， $\log q(x_t|x_0) - \log q(x_{t-1}|x_0)$  会逐项抵消：

$$\sum_{t=2}^T [\log q(x_t|x_0) - \log q(x_{t-1}|x_0)] = \log q(x_T|x_0) - \log q(x_1|x_0)$$

中间所有项  $q(x_2|x_0)$ ,  $q(x_3|x_0)$ ,  $\dots$ ,  $q(x_{T-1}|x_0)$  各出现一正一负，全部消掉。

## 第四步：合并整理

经过前三步，各项为：

$$\log p(x_T) + \underbrace{\log p_{\theta}(x_0|x_1)}_{t=1 \text{ 拆出}} + \sum_{t=2}^T \log p_{\theta}(x_{t-1}|x_t) - \sum_{t=2}^T \log q(x_{t-1}|x_t, x_0) - \log q(x_T|x_0) + \underbrace{\log q(x_1|x_0)}_{\text{伸缩残留}} - \underbrace{\log q(x_1|x_0)}_{t=1 \text{ 原始项}}$$

其中  $+\log q(x_1|x_0)$  (伸缩残留) 与  $-\log q(x_1|x_0)$  ( $t=1$  的原始项  $-\log q(x_1|x_0)$ ) 正负抵消。剩余项按三组合并：

$$\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} = \underbrace{\log \frac{p(x_T)}{q(x_T|x_0)}}_{L_T} + \sum_{t=2}^T \underbrace{\log \frac{p_{\theta}(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}}_{L_{t-1}} - \underbrace{\log p_{\theta}(x_0|x_1)}_{L_0}$$

对  $q$  取期望并加负号后， $-\mathbb{E}_q[\log(p/q)] = D_{\text{KL}}(q||p)$  (KL 散度的定义)，即得最终分解。

$$\begin{aligned} \text{ELBO} &= \underbrace{D_{\text{KL}}(q(x_T|x_0) || p(x_T))}_{L_T} \\ &\quad + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1}|x_t, x_0) || p_{\theta}(x_{t-1}|x_t))}_{L_{t-1}} \\ &\quad - \underbrace{\mathbb{E}_q[\log p_{\theta}(x_0|x_1)]}_{L_0} \end{aligned}$$

| 项                                                                         | 含义                                                           | 训练中的角色                                          |
|---------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------|
| $L_T = D_{\text{KL}}(q(x_T x_0)    p(x_T))$                               | 前向终点 $q(x_T x_0) \approx N(0, I)$ 与先验 $p(x_T) = N(0, I)$ 的匹配 | 常数 ( $\beta$ schedule 固定后不含可学习参数, $\approx 0$ ) |
| $L_{t-1} = D_{\text{KL}}(q(x_{t-1} x_t, x_0)    p_{\theta}(x_{t-1} x_t))$ | 模型的去噪一步 $p_{\theta}$ 与真实的反向后验 $q$ 的匹配                        | 核心项 (共 $T-1$ 项, 训练的主要目标)                        |
| $L_0 = -\mathbb{E}_q[\log p_{\theta}(x_0 x_1)]$                           | 最终一步的重建损失                                                    | 通常与 $L_{t-1}$ 统一处理                              |

### 2.4.2 核心项 $L_{t-1}$ 的计算

$$L_{t-1} = D_{\text{KL}}(q(x_{t-1}|x_t, x_0) || p_{\theta}(x_{t-1}|x_t))$$

这个 KL 散度衡量的是：**真实的反向后验** (2.3.4 节推导出来的) 与**模型学到的反向分布**之间的差距。

- 真实后验**  $q(x_{t-1}|x_t, x_0)$ ：高斯，均值  $\tilde{\mu}_t(x_t, x_0)$ ，方差  $\tilde{\beta}_t$  (已推导)
- 模型分布**  $p_{\theta}(x_{t-1}|x_t)$ ：也设定为高斯，**方差固定为  $\tilde{\beta}_t$**  (与真实后验相同)，只让模型学**均值**  $\mu_{\theta}(x_t, t)$

**为什么方差可以固定不学？** Ho et al. 2020 发现方差对生成质量影响不大，固定为  $\tilde{\beta}_t$  已经够好 (后来的 Improved DDPM 确实学了方差，但提升有限)。固定方差大大简化了问题——从“学一个分布”变成“学一个均值”。

两个方差相同的高斯之间的 KL 散度就是均值差的平方 (标准结论)：

上一步的损失表示  $\mu_t(x_t, t) \approx \tilde{\mu}_t(x_t, t)$ 。因此 2.2.2 节的结论，真实后验均值可以写成  $\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right)$ ，其中只有  $\epsilon$  是未知的。所以我们让模型也按这个结构来参数化——把未知的  $\epsilon$  替换成网络的预测  $\epsilon_\theta(x_t, t)$ ：

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

代入  $L_{t-1}$  的表达式， $x_t$  的项完全抵消，只剩噪声的差：

$$L_{t-1} = \frac{\beta_t^2}{2} \left( \tilde{\mu}_t - \mu_\theta(x_t, t) \right)^2 = \frac{\beta_t^2}{2} \left( \epsilon - \epsilon_\theta(x_t, t) \right)^2$$

#### 2.4.4 简化损失（Ho et al. 2020 的关键发现）

Ho et al. 发现，**去掉前面的时间步权重系数**，直接用简化损失训练效果更好：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon \sim \mathcal{N}(0, I), t \sim U\{1, T\}} \left[ \left( \epsilon - \epsilon_\theta(x_t, t) \right)^2 \right]$$

其中  $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon$ 。

##### 公式拆解：

- **下标**是三次采样： $x_0$ （从数据集随机取一张图）、 $\epsilon \sim \mathcal{N}(0, I)$ （随机生成纯噪声）、 $t \sim U\{1, T\}$ （随机选一个时间步）
- **中间步骤**：用闭式解  $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon$  一步构造出加噪图  $x_t$
- **内层**  $\left( \epsilon - \epsilon_\theta(x_t, t) \right)^2$ ：真实噪声  $\epsilon$ （我们自己加的，已知）与网络预测  $\epsilon_\theta$ （给定  $x_t, t$  后的输出）之间的 MSE
- **外层**  $\mathbb{E}[\cdot]$ ：对所有图片、时间步、噪声取期望（训练时用随机采样近似）

翻译成训练流程：随机取一张图 → 随机选时间步 → 随机加噪声 → 让网络猜噪声 → 猜错就扣分。

##### 为什么去掉权重反而更好？

- 原始 VLB 权重在  $t$  小时很大（因为  $\tilde{\beta}_t$  小），在  $t$  大时很小
- 简化损失对所有时间步权重一致，实际上给了高噪声级别（大  $t$ ）更多关注
- 高噪声级别负责全局结构，低噪声级别负责细节。均匀权重在实践中产生更好的生成质量

#### 2.5 三种等价的预测目标——为什么等价？

**问题背景**：上一节推导出的损失函数是  $\left( \epsilon - \epsilon_\theta(x_t, t) \right)^2$ ，即让网络预测噪声  $\epsilon$ 。但实际上，让网络预测**原始数据**  $x_0$  或预测**速度**  $v$  也能达到完全等价的效果。为什么？因为  $\epsilon$ 、 $x_0$ 、 $v$  三者之间可以通过**确定性的线性变换**互相转化——知道其中任何一个，就能立刻算出另外两个。

##### 2.5.1 核心关系： $\epsilon$ 、 $x_0$ 、 $v$ 之间的互转

一切都源于前向闭式解这一个等式（2.2.2 节）：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon$$

给定  $x_t$ （已知），这个等式将  $x_0$  和  $\epsilon$  **绑定**在一起——知道一个就能算出另一个：

**v 的来源：**注意到  $\sqrt{\bar{\alpha}_t}^2 + \sqrt{1-\bar{\alpha}_t}^2 = 1$ ，所以可以令  $\cos\theta = \sqrt{\bar{\alpha}_t}$ ， $\sin\theta = \sqrt{1-\bar{\alpha}_t}$ ，前向过程变成  $x_t = \cos\theta \cdot x_0 + \sin\theta \cdot \epsilon$ ，这是  $x_0$  和  $\epsilon$  在“信号-噪声”平面上角度为  $\theta$  的旋转插值。 $v$  正是  $x_t$  对  $\theta$  的导数  $\frac{dx_t}{d\theta} = -\sin\theta \cdot x_0 + \cos\theta \cdot \epsilon$ ，即插值路径上的切线方向。这保证了  $v$  在任何  $t$  处幅度都有界（系数平方和恒为 1），避免了  $\epsilon$  预测在  $t \rightarrow 0$  和  $x_0$  预测在  $t \rightarrow T$  时各自的数值不稳定。

所以三者知一推二，预测哪个都包含同样的信息量。

### 2.5.2 等价性证明： $\epsilon$ 预测 $\Leftrightarrow x_0$ 预测

**目标：**证明  $\|\epsilon - \epsilon_\theta\|^2$  和  $\|x_0 - x_{0,\theta}\|^2$  只差一个常数因子。

假设网络预测的噪声是  $\epsilon_\theta$ ，那么由上面的互转公式，它隐含地预测了  $x_0$ ：

$$x_{0,\theta} = \frac{x_t - \sqrt{1-\bar{\alpha}_t} \epsilon_\theta}{\sqrt{\bar{\alpha}_t}}$$

同理，真实的  $x_0$  对应真实的  $\epsilon$ ：

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

两者做差：

$$x_0 - x_{0,\theta} = \frac{\sqrt{1-\bar{\alpha}_t} (\epsilon - \epsilon_\theta)}{\sqrt{\bar{\alpha}_t}}$$

$x_t$  项消掉了（和 2.4.3 节相同的原因），取范数平方：

$$\|x_0 - x_{0,\theta}\|^2 = \frac{1-\bar{\alpha}_t}{\bar{\alpha}_t} \|\epsilon - \epsilon_\theta\|^2$$

反过来：

$$\|\epsilon - \epsilon_\theta\|^2 = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} \|x_0 - x_{0,\theta}\|^2$$

**结论：**两个损失只差一个因子  $\frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}$ （即信噪比 SNR），对于固定的  $t$  这是常数。在简化损失  $\mathcal{L}_{\text{simple}}$  中本身就丢掉了时间步权重，所以最小化  $\|\epsilon - \epsilon_\theta\|^2$  和最小化  $\|x_0 - x_{0,\theta}\|^2$  在优化方向上完全等价。

### 2.5.3 等价性证明： $\epsilon$ 预测 $\Leftrightarrow v$ 预测

**目标：**证明  $\|v - v_\theta\|^2$  和  $\|\epsilon - \epsilon_\theta\|^2$  只差一个正常数因子，因此有相同的最优解。

**第一步：将  $v$  表示为  $\epsilon$  的函数。**

速度定义为  $v = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1-\bar{\alpha}_t} x_0$ 。将  $x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$  代入：

$$v = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1-\bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1-\bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

展开第二项：

$$v = \frac{\bar{\alpha}_t + (1 - \bar{\alpha}_t)\sqrt{\bar{\alpha}_t}}{\epsilon} \cdot \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \cdot x_t = \frac{1}{\sqrt{\bar{\alpha}_t}} \cdot \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \cdot x_t$$

这是  $v = A\epsilon + b$  的仿射变换形式，其中  $A = \frac{1}{\sqrt{\bar{\alpha}_t}}$ （正常数）， $b = -\frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \cdot x_t$ （与  $\epsilon$  无关的常量）。

第二步：做差消去常数项

网络预测  $\epsilon_{\theta}$  时，隐含预测的速度  $v_{\theta}$  也服从同样的映射。做差时  $b$  项（含  $x_t$ ）完全相消：

$$v - v_{\theta} = \frac{1}{\sqrt{\bar{\alpha}_t}}(\epsilon - \epsilon_{\theta})$$

取范数平方：

$$\|v - v_{\theta}\|^2 = \frac{1}{\bar{\alpha}_t} \|\epsilon - \epsilon_{\theta}\|^2$$

**结论：**两个损失只差因子  $\frac{1}{\bar{\alpha}_t}$ ，对于固定的  $t$  这是正常数。正常数倍不改变梯度为零的位置（ $\nabla_{\theta}(c \cdot L) = c \cdot \nabla_{\theta} L$ ），因此  $\|v - v_{\theta}\|^2$  与  $\|\epsilon - \epsilon_{\theta}\|^2$  有相同的最优解  $\theta^*$ 。

2.5.4 三种目标的对比

|            | $\epsilon$ 预测                                                                                                                         | $x_0$ 预测                                                                                                                        | $v$ 预测                                   |
|------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 网络输出       | 预测加入的噪声                                                                                                                               | 预测去噪后的原图                                                                                                                        | 预测信号与噪声的“旋转”                             |
| 损失函数       | $\ \epsilon - \epsilon_{\theta}\ ^2$                                                                                                  | $\ x_0 - x_{0,\theta}\ ^2$                                                                                                      | $\ v - v_{\theta}\ ^2$                   |
| $t$ 小（噪声少） | 预测目标 $\epsilon$ 很小，信号弱                                                                                                                | 预测目标 $x_0$ 接近 $x_t$ ，容易                                                                                                         | 稳定                                       |
| $t$ 大（噪声多） | 预测目标 $\epsilon$ 接近 $x_t$ ，容易                                                                                                          | 预测目标 $x_0$ 与 $x_t$ 差异大，困难                                                                                                       | 稳定                                       |
| 数值稳定性      | $t \rightarrow 0$ 时 $\sqrt{1 - \bar{\alpha}_t} \rightarrow 0$ ，噪声信号微弱，损失权重 $\bar{\alpha}_t / (1 - \bar{\alpha}_t) \rightarrow \infty$ | $t \rightarrow T$ 时 $\sqrt{\bar{\alpha}_t} \rightarrow 0$ ，信号淹没，损失权重 $(1 - \bar{\alpha}_t) / \bar{\alpha}_t \rightarrow \infty$ | 两端都稳定                                    |
| 代表工作       | DDPM (Ho 2020)                                                                                                                        | Ramesh et al. (DALL-E)                                                                                                          | Salimans & Ho 2022, Stable Diffusion v2+ |

**$v$  预测的直觉：**可以把前向过程看作在“信号-噪声”平面上的旋转—— $x_0$  沿一个方向衰减， $\epsilon$  沿另一个方向增长， $v$  就是这个旋转的“角速度”。它天然地平衡了信号和噪声两端，避免了  $\epsilon$  预测和  $x_0$  预测各自在极端  $t$  处的数值问题。

**数值稳定性的详细解释：**这里的“不稳定”**不是指某个公式的分母除以零**，而是指**损失函数在极端  $t$  处的隐含权重失衡**。

回忆 2.5.2 节的等价性公式： $\|v - v_{\theta}\|^2 = \frac{1}{\bar{\alpha}_t} \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \|x_0 - x_{0,\theta}\|^2$ ，其中  $\frac{1}{\bar{\alpha}_t} \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}$  是信噪比 SNR。

- $\epsilon$  预测在  $t \rightarrow 0$  时不稳定：** $t \rightarrow 0$  时  $\bar{\alpha}_t \rightarrow 1$ ， $1 - \bar{\alpha}_t \rightarrow 0$ ，信噪比  $\text{SNR} = \frac{1}{\bar{\alpha}_t} \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \rightarrow \infty$ 。这意味着  $x_0$  上一个很小的误差  $\|x_0 - x_{0,\theta}\|^2$ ，在  $\epsilon$  空间会被放大为  $\text{SNR}$  倍那么大的损失值。训练时，低噪声样本产生巨大的梯度，主导了整个训练过程。直觉： $t$  接近 0 时  $x_t \approx x_0$ ，噪声  $\epsilon$  对  $x_t$  的贡献是  $\sqrt{1 - \bar{\alpha}_t} \epsilon \approx$

信息已淹没，网络要从纯噪声中预测清晰原图——尤其中生。

- **v 预测两端都稳定**： $v = \sqrt{\bar{\alpha}_t} \sqrt{\epsilon - \sqrt{1 - \bar{\alpha}_t}} x_0$ ，系数满足  $(\sqrt{\bar{\alpha}_t})^2 + (\sqrt{1 - \bar{\alpha}_t})^2 = 1$ 。v 预测的损失  $\|v - v_{\theta}\|^2 = \frac{1}{\bar{\alpha}_t} \sqrt{\epsilon - \epsilon_{\theta}}^2$ ，权重因子  $\frac{1}{\bar{\alpha}_t}$  虽然在  $t \rightarrow T$  时增大，但增长速率远低于 SNR 的极端值，同时在  $t \rightarrow 0$  时不会爆炸 ( $\frac{1}{\bar{\alpha}_t} \rightarrow 1$ )。更关键的是，v 的反推公式  $x_0 = \sqrt{\bar{\alpha}_t} x_t - \sqrt{1 - \bar{\alpha}_t} v$ 、 $\epsilon = \sqrt{1 - \bar{\alpha}_t} x_t + \sqrt{\bar{\alpha}_t} v$  中**没有除法**，采样过程也不会数值爆炸。

## 2.6 采样过程详解

### 2.6.1 DDPM 采样（随机，T 步）

根据反向后验  $p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \sigma_t^2 I)$ ：

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left( x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \right) \sqrt{\epsilon_{\theta}(x_t, t)} + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

这是一个**随机微分方程 (SDE)** 的离散化，每一步都加入新的随机噪声  $z$ 。

### 2.6.2 DDIM 采样（确定性，可加速）

DDPM 采样需要走完所有  $T$  步（如 1000 步），每步一次网络推理，速度很慢。而且每步都加随机噪声  $\sigma_t z$ ，同一个  $x_T$  每次生成的  $x_0$  都不同。

Song et al. 2020 (DDIM) 的关键洞察：反向过程**不一定是随机的马尔可夫链**。我们唯一需要保证的约束只有一个——前向过程的边缘分布  $q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$  不变。只要这个约束满足，训练好的  $\epsilon_{\theta}$  仍然有效。

**DDIM 的两步走策略：**

**第一步：用网络预测估计  $\hat{x}_0$** （从  $\epsilon_{\theta}$  反推  $x_0$ ，即 2.5.1 节的互转公式）：

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

**第二步：从  $\hat{x}_0$  “重新加噪”到  $x_{t-1}$ 。**前向过程告诉我们  $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ ，那么  $x_{t-1}$  也满足类似的关系——只需把时间步从  $t$  换成  $t-1$ ，用同一个  $\hat{x}_0$  和同一个  $\epsilon_{\theta}$ ：

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_{\theta}(x_t, t)$$

**为什么第二步可以用同一个  $\epsilon_{\theta}$  而不重新采样噪声？**这正是 DDIM “确定性”的来源。DDPM 在每步加一个新的随机噪声  $z$ ，而 DDIM 认为：既然我们已经从网络得到了对噪声的最优估计  $\epsilon_{\theta}$ ，就不需要额外引入新的随机性。用同一个  $\epsilon_{\theta}$  保证了过程的确定性和一致性。

**为什么可以跳步？**公式中只出现  $\bar{\alpha}_t$  和  $\bar{\alpha}_{t-1}$ （累积系数），不需要单步衰减系数  $\alpha_t$ 。所以  $t$  和  $t-1$  不必是相邻时间步——可以定义一个子序列  $\tau = [T, T-k, T-2k, \dots, 0]$ ，每步直接跳  $k$  个时间步。例如  $T = 1000$  但只用 50 步采样，**速度提升 20 倍**。

深入浅出——增量 VC 坐标

已赞同 1

添加评论

★ 收藏

♥ 喜欢

➔ 分享

📄 申请转载

✦

...

加入新随机噪声  $z$ ，这些噪声逐步累积——跳步等于跳过了应该加的噪声，最终分布就不对了。

- **DDIM** 公式只含  $\bar{\alpha}_t$ 、 $\bar{\alpha}_{t-1}$ （**累积坐标**）：描述的是“海拔高度”而非“台阶高度”。只要知道当前海拔（ $\bar{\alpha}_t$ ）和目标海拔（ $\bar{\alpha}_{t-1}$ ），就能直接传送，不关心中间经过了多少级台阶。

更具体地说，前向闭式解  $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$  中，**任意两个时刻之间没有依赖关系**——只要知道  $x_0$  和  $\epsilon$ ，就能直接算出任意时刻的  $x_t$ 。DDIM 正是利用了这一点：每步先估计  $\hat{x}_0$  和  $\epsilon_{\theta}$ ，然后直接跳到任意目标时刻。

**跳步采样的具体过程**（以  $T = 1000$ 、5 步采样为例）：

定义子序列  $\tau = [1000, 800, 600, 400, 200, 0]$ （6 个时间点，形成 **5 个间隔**，即 5 步迭代），每步做的事情完全相同：

| 迭代    | 从 → 到                          | 操作                                |
|-------|--------------------------------|-----------------------------------|
| 第 1 步 | $x_{1000} \rightarrow x_{800}$ | 网络预测 $\hat{x}_0$ ，“重新加噪”到 $t=800$ |
| 第 2 步 | $x_{800} \rightarrow x_{600}$  | 同上，“重新加噪”到 $t=600$                |
| 第 3 步 | $x_{600} \rightarrow x_{400}$  | 同上                                |
| 第 4 步 | $x_{400} \rightarrow x_{200}$  | 同上                                |
| 第 5 步 | $x_{200} \rightarrow x_0$      | 最后一步直接得到 $\hat{x}_0$              |

为什么不一步直接到  $x_0$ ？因为网络在高噪声下对  $x_0$  的估计不够准确，分多步逐渐降噪可以让每步的估计越来越精确。步数是精度与速度的权衡。

**一句话总结**：DDIM 把反向过程从“逐步转移”重写成了“先猜终点、再去中间站”，而“去中间站”这个操作只需要目标时刻的累积系数  $\bar{\alpha}$ ，跟路径无关。

**DDPM vs DDIM 对比：**

|       | DDPM                                                          | DDIM                                                                                                     |
|-------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 每步公式  | $x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z, z \sim N(0, I)$ | $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_{\theta}$ （无随机项） |
| 随机性   | 有（每步加新噪声）                                                     | 无（确定性，可复现）                                                                                               |
| 步数    | 必须 $T$ 步                                                      | 可跳步（如 50 步）                                                                                              |
| 隐空间插值 | 不支持                                                           | 支持（ $x_T \leftrightarrow x_0$ 是确定性双射）                                                                    |

**本质**：DDIM 去掉了每步的随机噪声  $\sigma_t z$ ，把随机过程变成了确定性过程。用数学语言说，就是把 SDE（随机微分方程）转化成了 ODE（常微分方程）。如果你对 SDE/ODE 不熟悉，不用担心——2.7.3 节会从零解释这些概念，并揭示 DDIM 能工作的深层数学原因（剧透：Song et al. 2021 证明了确定性 ODE 和随机 SDE 能产生**完全相同的概率分布**，所以去掉随机项不影响生成质量）。这也是连接 Diffusion 和 Flow Matching 的重要桥梁。

## 2.7 Score Matching 视角——扩散模型在学什么？

**本节的作用**：前面我们从“预测噪声  $\epsilon$ ”的角度理解了扩散模型。本节揭示一个更深层的视角：**扩散模型实际上在学习数据分布的梯度场（score function）**。这个视角是理解 DDIM 为什么能工作、以及 Diffusion 和 Flow Matching 如何统一的关键。



先拆解这个公式里的每个部分：

- $p(x)$ ：数据的概率密度函数。比如对于图像数据， $p(x)$  描述了“自然图像”在整个像素空间中的分布——某些像素组合（看起来像猫、像风景的）概率高，大部分随机像素组合概率极低。
- $\log p(x)$ ：取对数后的概率密度。
- $\nabla_x$ ：**对输入  $x$ （不是对参数  $\theta$ ）求梯度**。对于一张  $64 \times 64$  的图像（ $d = 64 \times 64 \times 3 = 12288$  维）， $\nabla_x \log p(x)$  是一个 12288 维的向量——**告诉你每个像素朝哪个方向调整，能让  $x$  变得“更像真实数据”**。

直觉理解：

想象数据分布  $p(x)$  是一个地形图，其中“海拔”代表概率密度。score function  $\nabla_x \log p(x)$  就是这个地形上每一点的**上坡方向**——一个向量，指向概率密度增大最快的方向。

- 在概率密度的**峰值**（数据聚集的地方）， $\text{score} \approx 0$ （已经在山顶了，没有更高的方向）
- 在概率密度的**低谷**（远离数据的地方）， $\text{score}$  指向最近的峰值（告诉你往哪走能找到数据）
- $\text{score}$  的**大小**表示密度变化的剧烈程度（梯度越大，坡越陡，离峰值越近）

**什么是“数据聚集的地方”？** 以猫图像数据集为例。每张  $64 \times 64 \times 3$  的图片是 12288 维空间中的一个点。这个空间里绝大多数坐标对应的是随机噪点——无意义的像素组合。但所有“橘猫正面照”的像素组合在空间中的坐标会彼此接近，形成一个**簇**；所有“黑猫侧面照”形成另一个簇。这些簇就是  $p(x)$  的峰值——“数据聚集的地方”。 $\text{score}$  在一张“鼻子有点歪的猫”上的值，就是在告诉你“把这几个像素往这边调，鼻子就正了，就更像真实的猫了”。

**与去噪的联系：** 现在想象你有一张加了噪声的模糊图  $x_t$ ，它偏离了“真实图像”的峰值。 $\text{score} \nabla_{x_t} \log p(x_t)$  告诉你“往哪个方向改像素能更接近真实图像”——这不就是**去噪的方向**吗？扩散模型的采样本质上就是：从纯噪声出发，沿着  $\text{score}$  的方向一步步走向概率密度的峰值。

**为什么用  $\log p(x)$  而不是  $p(x)$ ？** 取  $\log$  有两个好处：（1） $p(x)$  在高维空间极小（想象  $256^{12288}$  种可能的像素组合中，自然图像只占极微小的子集），取  $\log$  后数值更稳定；（2）更关键的是， $\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)}$ ——**自动做了归一化**。否则  $\nabla_x p(x)$  在高密度区域很大、低密度区域很小，幅度变化太剧烈。取  $\log$  后除以  $p(x)$  平衡了这个问题。

**为什么学  $\text{score}$  而不直接学  $p(x)$ ？** 直接学  $p(x)$  需要保证积分为 1（归一化常数  $Z$ ），这在高维空间极难计算。而  $\nabla_x \log p(x) = \nabla_x \log \frac{p(x)}{Z} = \nabla_x \log p(x)$ ——**对  $x$  求梯度时  $Z$  直接消失了**（ $\log Z$  是常数，梯度为零）。所以学  $\text{score}$  完全避开了归一化常数的问题。

### 2.7.2 扩散模型与 Score 的关系

**核心结论：** 训练好的噪声预测网络  $\epsilon_{\theta}(x_t, t)$  实际上在近似 score function：

$$\epsilon_{\theta}(x_t, t) \approx -\sqrt{1-\alpha_t} \nabla_{x_t} \log q(x_t)$$

也就是说：

$$\nabla_{x_t} \log q(x_t) \approx -\frac{\epsilon_{\theta}(x_t, t)}{\sqrt{1-\alpha_t}}$$

**预测噪声 = 学 score（差一个缩放因子）。**

**为什么预测噪声就等于学 score？直觉解释：**

已赞同 1

添加评论

★ 收藏

♥ 喜欢

➔ 分享

📄 申请转载

✦

...



知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知识堂

Q

D 直答



消息

私信

| 噪声 的时候，就等于在告诉你 在哪里能回到干净数据。

完整推导：

**第一步：**写出  $q(x_t)$  的 score。 $q(x_t)$  是所有可能的  $x_0$  加噪后得到  $x_t$  的边缘分布：

$$q(x_t) = \int q(x_t|x_0)q(x_0)dx_0$$

对  $x_t$  求梯度（用  $\nabla \log = \frac{\nabla}{\cdot}$  和积分交换求导）：

$$\nabla_{x_t} \log q(x_t) = \frac{\nabla_{x_t} q(x_t)}{q(x_t)} = \frac{\int \nabla_{x_t} q(x_t|x_0)q(x_0)dx_0}{q(x_t)}$$

把  $\frac{q(x_0)}{q(x_t)}$  凑成后验  $q(x_0|x_t) = \frac{q(x_t|x_0)q(x_0)}{q(x_t)}$ ：

$$\begin{aligned} &= \int \frac{\nabla_{x_t} q(x_t|x_0)}{q(x_t|x_0)} \cdot \frac{q(x_t|x_0)q(x_0)}{q(x_t)} dx_0 \\ &= \int \nabla_{x_t} \log q(x_t|x_0) \cdot q(x_0|x_t) dx_0 \\ &= \mathbb{E}_{q(x_0|x_t)} [\nabla_{x_t} \log q(x_t|x_0)] \end{aligned}$$

**这步的意义：**边缘分布  $q(x_t)$  的 score = 条件分布  $q(x_t|x_0)$  的 score 关于后验  $q(x_0|x_t)$  的期望。这里“score”就是对  $x_t$  求梯度  $\nabla_{x_t} \log q(\cdot)$ ，两边都是向量。这步的价值在于：左边的  $q(x_t) = \int q(x_t|x_0)q(x_0)dx_0$  是复杂的积分，直接求梯度算不了；右边的  $q(x_t|x_0)$  是已知的高斯分布，梯度有闭式解。**把算不了的转化成了算得了的。**

**第二步：**计算条件分布的 score。由于  $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ ，高斯分布的 score 有简单的公式  $\nabla_x \log \mathcal{N}(x; \mu, \sigma^2 I) = -\frac{x-\mu}{\sigma^2}$ ：

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{(1-\bar{\alpha}_t)}$$

由前向过程  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ ，有  $x_t - \sqrt{\bar{\alpha}_t}x_0 = \sqrt{1-\bar{\alpha}_t}\epsilon$ ，代入：

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{\sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{1-\bar{\alpha}_t}} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

**第三步：**合并。将第二步代入第一步：

$$\nabla_{x_t} \log q(x_t) = \mathbb{E}_{q(x_0|x_t)} \left[ -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}} \right] = -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \mathbb{E}_{q(x_0|x_t)} [\epsilon]$$

而训练  $\epsilon_\theta$  的目标  $\|\epsilon - \epsilon_\theta\|^2$  的最优解恰好是  $\epsilon_\theta = \mathbb{E}_{q(\epsilon|x_t)}[\epsilon]$ （L2 损失的最优解是条件期望），所以：

**为什么 L2 损失的最优解是条件期望？**

这是统计学的经典结论，下面逐步推导。

**问题：**找一个函数  $f(x)$ ，使得  $\mathbb{E}[(y - f(x))^2]$  最小。**第1步：引入条件期望。**定义  $\mu(x) = \mathbb{E}[y|x]$ ，即给定  $x$  后  $y$  的条件均值。**第2步：拆项。**把  $y - f(x)$  拆成两部分（第一项为随机波动，第二项为预测偏差）：

$$y - f(x) = (y - \mu) + (\mu - f(x))$$

**第3步：展开平方。**

$$\mathbb{E}[(y - f(x))^2] = \mathbb{E}[(y - \mu)^2] + \mathbb{E}[(\mu - f(x))^2] + 2\mathbb{E}[(y - \mu)(\mu - f(x))]$$

已赞同 1



添加评论



收藏



喜欢



分享



申请转载



...

$$\mathbb{E}[(y - \mu)(\mu - f(x)) \mid x] = (\mu - f(x)) \cdot \mathbb{E}[y - \mu \mid x]$$

而  $\mathbb{E}[y - \mu \mid x] = \mathbb{E}[y \mid x] - \mu = \mu - \mu = 0$ ，所以整个交叉项为零。

**第4步：得到最终形式。**

$$\mathbb{E}[(y - f(x))^2 \mid x] = \mathrm{Var}(y \mid x) + (\mu - f(x))^2$$

其中第一项  $\mathrm{Var}(y \mid x)$  为不可约噪声（与  $f$  无关），第二项  $(\mu - f(x))^2 \geq 0$  可优化。

- 第一项  $\mathrm{Var}(y \mid x)$  是数据本身的随机性，无论怎么选  $f$  都消不掉
- 第二项  $(\mu - f(x))^2 \geq 0$ ，当且仅当  $f(x) = \mu = \mathbb{E}[y \mid x]$  时取到最小值零

**结论：L2 损失的最优解  $f^*(x) = \mathbb{E}[y \mid x]$ ，即条件期望。**

**直觉类比：**让很多人从同一个位置投篮，落点有随机性。要预测“篮球落在哪”使预测误差最小，最优答案就是所有落点的平均值。L2 损失天然地选择“平均”作为最优解。

**回到扩散模型：**输入  $x_t$ （加噪图像）、目标  $\epsilon$ （噪声）。同一个  $x_t$  可能由不同的  $(x_0, \epsilon)$  组合产生，所以  $\epsilon_{\theta}^* = \mathbb{E}[\epsilon \mid x_t]$ 。后面的  $\mathbb{E}_q[\epsilon \mid x_t] = \mathbb{E}_q[x_0 \mid x_t](\epsilon)$  是因为给定  $x_t$  后  $\epsilon$  和  $x_0$  一一对应（ $\epsilon = \frac{x_t - \sqrt{1 - \alpha_t} x_0}{\sqrt{1 - \alpha_t}}$ ），对  $\epsilon$  求期望等价于对  $x_0$  求期望。

$$\nabla_{x_t} \log q(x_t) = -\frac{\epsilon_{\theta}^*(x_t, t)}{\sqrt{1 - \alpha_t}}$$

**结论：** $\epsilon_{\theta}(x_t, t) \approx -\sqrt{1 - \alpha_t} \nabla_{x_t} \log q(x_t)$ 。**预测噪声就是在学 score。**

**为什么这个结论重要？** 它把两个看似不同的研究方向统一了：（1）DDPM 那边说的是“预测噪声  $\epsilon$ ”；（2）Score Matching 那边说的是“学习 score function  $\nabla_x \log p(x)$ ”。实际上它们做的是完全相同的事情，只差一个缩放因子  $\sqrt{1 - \alpha_t}$ 。这个统一视角是 Score SDE 框架和后来 Flow Matching 的理论基础。

### 2.7.3 Probability Flow ODE

#### 前置知识：什么是 ODE 和 SDE

**ODE（常微分方程，Ordinary Differential Equation）** 描述“变化率”的方程。最简单的例子：你开车，速度是  $v(t)$ ，位置是  $x(t)$ ，那么：

$$\frac{dx}{dt} = v(t)$$

含义是：**知道每个时刻的速度（变化率），就能从起点算出任意时刻的位置。** ODE 的关键特点是**确定性的**——给定初始位置  $x(0)$ ，轨迹完全确定，没有随机性。

**SDE（随机微分方程，Stochastic Differential Equation）** 在 ODE 基础上加了一个随机扰动项：

$$dx = \underbrace{f(x, t) dt}_{\text{确定性漂移 (drift)}} + \underbrace{g(t) dw}_{\text{随机扰动 (diffusion)}}$$

**类比：**你开车（确定性漂移  $f$ ），但路上有风不断随机推你（随机扰动  $g \cdot dw$ ）。即使从同一个起点出发，每次跑的轨迹都不一样。 $dw$  是布朗运动——可以理解为“每一小步都随机偏移一点”，方向和大小完全随机。

## 扩散模型中的 SDE

DDPM 的**前向过程**（加噪）本质上就是一个 SDE：

- 确定性部分  $f(x,t)$ ：让图像的信号逐渐衰减
- 随机部分  $g(t)\backslash dw$ ：不断往图像上加随机噪声

DDPM 的**反向过程**（去噪/采样）也是一个 SDE——DDPM 采样时每步都加了随机噪声  $\backslash\sigma_t z$ ，这就是反向 SDE 中的随机项。

$$dx = f(x,t)\backslash dt + g(t)\backslash dw$$

其中  $f(x,t)$  是漂移项（drift）， $g(t)$  是扩散系数（diffusion）， $dw$  是布朗运动（随机项）。

## Probability Flow ODE 的核心定理

Song et al. 2021 (Score SDE) 证明了一个很强的定理：**对于任意 SDE，都存在一个确定性的 ODE（称为概率流 ODE），使得两者在每个时刻  $t$  的概率分布  $p_t(x)$  完全相同。**

$$\backslash\frac{dx}{dt} = f(x,t) - \backslash\frac{1}{2}g(t)^2 \backslash\nabla_x \log p_t(x)$$

**这句话到底在说什么？** 先回顾背景：在扩散模型中，我们关心的不是“某一个粒子去了哪里”，而是**整体的概率分布怎么变化**。比如前向过程： $t=0$  时  $p_0(x)$  是真实图像的分布（复杂、多峰）， $t=T$  时  $p_T(x)$  是纯高斯噪声（简单、一个大鼓包）。整个过程就是  $p_0 \rightarrow p_1 \rightarrow \cdots \rightarrow p_T$ ，每个时刻都有一个概率分布。

现在有**两种方式**都能实现“从  $p_0$  演化到  $p_T$ ”：

**方式一：SDE（随机的）。** 每个样本按照  $dx = f(x,t)\backslash dt + g(t)\backslash dw$  演化。因为有随机项  $dw$ ，同一个起点跑 100 次会得到 100 条不同的轨迹。但如果你跑 **100 万次**，统计每个时刻所有样本的分布， $t=5$  时刻它们的分布是  $p_5(x)$ ， $t=10$  时刻是  $p_{10}(x)$ 。

**方式二：ODE（确定性的）。** 每个样本按照  $\backslash\frac{dx}{dt} = f(x,t) - \backslash\frac{1}{2}g(t)^2 \backslash\nabla_x \log p_t(x)$  演化。没有随机项，同一个起点跑 100 次都是完全相同的一条轨迹。但如果你从  $p_0(x)$  中抽 **100 万个起点**，让每个点沿各自的确定性轨迹走，统计  $t=5$  时刻所有点的分布——**和 SDE 得到的  $p_5(x)$  一模一样！**

**沙堆类比：**要把一堆沙子从一个形状（ $p_0$ ：复杂图案）变成另一个形状（ $p_T$ ：均匀铺平）：

|           | SDE 方式            | ODE 方式               |
|-----------|-------------------|----------------------|
| 做法        | 不断随机抖动桌子，沙粒各自随机弹跳 | 用精确的推沙器，按计算好的路径推每粒沙子 |
| 单粒沙子      | 每次轨迹不同            | 每次轨迹相同               |
| 最终沙堆形状    | 铺平了               | 也铺平了，而且形状一样          |
| 中间任意时刻的形状 | $p_t$             | 也是 $p_t$ ，完全相同       |

**确定性修正项的直觉：**SDE 中随机项  $g(t)\backslash dw$  的宏观效果是让粒子**扩散开**（从密集处向稀疏处弥散）。ODE 中没有随机项，怎么模拟同样的“扩散效果”？靠 score function  $\backslash\nabla_x \log p_t(x)$ 。这个向量指向概率密度增大的方向，乘以负号后就**把粒子从密集处推向稀疏处**——和随机扩散的宏观效果完全一样，但每个粒子走的是确定性路径。

## 为什么 Probability Flow ODE 重要

**从 DDIM 的视角：**在扩散模型中，我们关心的**不是前向过程**，而是**反向过程**（去噪）生成图像。因此，我们更关心 ODE 的确定性，因为它更容易优化。

已赞同 1

添加评论

收藏

喜欢

分享

申请转载

...

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息

私信

过程中的随机项  $\sqrt{\sigma_t} z$ （即设  $\sigma_t = 0$ ），变成确定性条件——这正好对应从“反向 SDE”切换到“概率流 ODE”。

**与 Flow Matching 的联系：**概率流 ODE 的形式  $\frac{dx}{dt} = v_t(x)$ （某个速度场）和 Flow Matching 的 ODE  $\frac{d\phi_t(x)}{dt} = v_{\theta}(\phi_t(x), t)$  在结构上完全相同。这就是连接 Diffusion 和 Flow Matching 的关键桥梁——**两者都在学一个速度场/向量场来驱动 ODE**，只是训练方法不同。

**采样加速：**确定性 ODE 可以用更高阶的 ODE 求解器（如 RK4），跳更大的步，比 SDE 采样更快。这是 DDIM 比 DDPM 快的根本原因。

## 2.8 特点

- [+] 生成质量极高（图像、音频、视频）
- [+] 训练稳定（不需要对抗训练，不会 mode collapse）
- [+] 灵活（classifier-free guidance、ControlNet 等条件控制）
- [-] 采样慢（需要多步迭代，即使加速后仍比 GAN 慢）
- [-] 噪声调度设计需要调参
- [-] 数学框架较重（SDE  $\rightarrow$  VLB  $\rightarrow$  简化目标，经过多层近似）

## 2.9 应用

DALL-E 2/3、Stable Diffusion 1.x/2.x、Imagen、Sora（视频）、AudioLDM（音频）

## 2.10 核心速记卡

看完 Diffusion 后，闭上文档能说出这些，就算掌握了。

**一句话总结：**前向加噪有闭式解  $\rightarrow$  反向去噪给定  $x_0$  后可算  $\rightarrow$  用  $\epsilon$  替换  $x_0 \rightarrow$  训练网络预测噪声  $\epsilon$ 。

**核心逻辑链（6 步）：**

1. 前向闭式解：  $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$ （一步跳到任意  $t$ ）  
 $\downarrow$
2. 反向  $q(x_{t-1}|x_t)$  不可算（分母需要对整个数据分布积分）  
 $\downarrow$
3. 给定  $x_0$  后，  $q(x_{t-1}|x_t, x_0)$  变成已知高斯（三个高斯配方）  
 $\downarrow$
4. 后验均值  $\mu_t$  只依赖  $x_t$  和  $\epsilon \rightarrow \epsilon$  是唯一未知量  
 $\downarrow$
5. 训练网络  $\epsilon_{\theta}(x_t, t)$  预测噪声  $\epsilon$   
 $\downarrow$
6. 损失：  $\|\epsilon - \epsilon_{\theta}\|^2$ （从 VLB  $\rightarrow$  KL  $\rightarrow$  MSE 逐步简化而来）

**关键公式（4 个）：**

| 公式                                                                                  | 含义                              |
|-------------------------------------------------------------------------------------|---------------------------------|
| $x_t = \sqrt{\alpha_t} x_0 + \sqrt{(1-\alpha_t)} \epsilon$                          | 前向闭式解（一切的基础）                    |
| $\mu_t = (1/\sqrt{\alpha_t})(x_t - \beta_t/\sqrt{(1-\alpha_t)}\epsilon)$            | 反向后验均值（只依赖 $x_t$ 和 $\epsilon$ ） |
| $L = \ \epsilon - \epsilon_{\theta}(x_t, t)\ ^2$                                    | 简化损失                            |
| $\epsilon_{\theta} \approx -\sqrt{(1-\alpha_t)} \cdot \nabla_{\{x_t\}} \log q(x_t)$ | 预测噪声 = 学 score                  |

**DDPM vs DDIM 一句话区分：**

• 已赞同 1 • 添加评论 • 收藏 • 喜欢 • 分享 • 申请转载 • ...

但早期的做法，Diffusion 的路径是弯曲的（\(\gamma\_t\|x\_0 - x\_t\|^2\|x\_t - x\_1\|^2\) 非线性），而无论多少维都不能精确积分。Flow Matching 的核心改进就是把路径拉直——用线性插值  $(1-t)x_0 + tx_1$  替代非线性插值。

## 2.11 PyTorch 实现

**关于去噪骨干架构：**下面的代码使用经典的 U-Net 结构以保持教学简洁。在现代实践中（SD3、Flux、Sora），U-Net 已被 **DiT（Diffusion Transformer）** 替代——用 ViT 式 Patch Embedding + Transformer Block 替代卷积编解码器，获得更好的 Scaling Law 和全局注意力。DiT 的完整架构原理、adaLN-Zero 条件注入机制、核心代码和 Scaling Law 数据详见 5.3 节。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

# =====
# Part 1: 时间步嵌入 — 告诉网络“现在噪声有多大”
# =====

class SinusoidalTimeEmbedding(nn.Module):
    """
    把离散的时间步  $t \in \{0, 1, \dots, T-1\}$  编码成连续向量。

    为什么需要？网络必须知道当前是哪个时间步，因为  $t=10$ （几乎无噪声）和
     $t=990$ （几乎全噪声）需要完全不同的去噪策略。直接传整数  $t$  效果差，用
    正弦/余弦编码能让网络更容易区分不同的时间步——和 Transformer 的位置
    编码原理完全相同。

    编码公式（与 Transformer position encoding 一致）：
         $PE(t, 2i) = \sin(t / 10000^{(2i/d)})$ 
         $PE(t, 2i+1) = \cos(t / 10000^{(2i/d)})$ 
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        device = t.device
        half_dim = self.dim // 2
        #  $\log(10000) / (d/2 - 1)$  是频率的衰减系数
        emb = math.log(10000) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb)
        #  $t[:, None] * emb[None, :] \rightarrow (batch, half\_dim)$ ，每个频率一个值
        emb = t.float()[ :, None] * emb[None, :]
        # 拼接 sin 和 cos  $\rightarrow (batch, dim)$ 
        emb = torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1)
        return emb

# =====
# Part 2: 噪声预测网络（U-Net 骨架）
# =====
```

已赞同 1



添加评论

★ 收藏

❤ 喜欢

➔ 分享

📄 申请转载



简化版 UNet，用于演示 DDPM 的核心流程。

结构：Encoder(下采样) → Bottleneck → Decoder(上采样 + skip connection)  
每一层都注入时间步嵌入，让网络知道当前的噪声水平。

实际生产中（Stable Diffusion 等）会用更复杂的 UNet：

- 加入 Attention 层（self-attention + cross-attention）
- 更多分辨率层级（64→32→16→8）
- Group Normalization、残差连接等

这里为了可读性做了大幅简化。

"""

```
def __init__(self, in_channels=1, base_channels=64, time_emb_dim=128):
    super().__init__()

    # 时间步编码：整数 t → 128 维向量 → 各层用线性层投影到对应通道数
    self.time_mlp = nn.Sequential(
        SinusoidalTimeEmbedding(time_emb_dim),
        nn.Linear(time_emb_dim, time_emb_dim),
        nn.SiLU(),
    )

    # ----- Encoder（下采样） -----
    ch = base_channels # 64
    self.enc1 = self._block(in_channels, ch) # (B,1,28,28) → (B,64,28,28)
    self.time_proj1 = nn.Linear(time_emb_dim, ch)
    self.pool1 = nn.MaxPool2d(2) # → (B,64,14,14)

    self.enc2 = self._block(ch, ch * 2) # → (B,128,14,14)
    self.time_proj2 = nn.Linear(time_emb_dim, ch * 2)
    self.pool2 = nn.MaxPool2d(2) # → (B,128,7,7)

    # ----- Bottleneck -----
    self.bottleneck = self._block(ch * 2, ch * 4) # → (B,256,7,7)
    self.time_proj_bn = nn.Linear(time_emb_dim, ch * 4)

    # ----- Decoder（上采样 + skip connection） -----
    self.up2 = nn.ConvTranspose2d(ch * 4, ch * 2, kernel_size=2, stride=2) #
    self.dec2 = self._block(ch * 4, ch * 2) # 输入 ch*4 因为 cat 了 skip
    self.time_proj3 = nn.Linear(time_emb_dim, ch * 2)

    self.up1 = nn.ConvTranspose2d(ch * 2, ch, kernel_size=2, stride=2) #
    self.dec1 = self._block(ch * 2, ch) # 输入 ch*2 因为 cat 了 skip
    self.time_proj4 = nn.Linear(time_emb_dim, ch)

    # 最终输出：通道数 = 输入通道数，预测的是噪声 ε
    self.final = nn.Conv2d(ch, in_channels, kernel_size=1)

def _block(self, in_ch, out_ch):
    """Conv → GroupNorm → SiLU → Conv → GroupNorm → SiLU"""
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, 3, padding=1),
        nn.GroupNorm(8, out_ch),
        nn.SiLU(),
        nn.Conv2d(out_ch, out_ch, 3, padding=1),
        nn.GroupNorm(8, out_ch),
        nn.SiLU(),
    )
```

已赞同 1



添加评论



收藏



喜欢



分享



申请转载





知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息

私信

`def forward(self, x): # 前向采样和预测噪声` `t: (B,) - 时间步, 整数 0 ~ T-1` `输出:` `(B, C, H, W) - 预测的噪声  $\epsilon_\theta(x_t, t)$`  `"""` `# 时间步编码` `t_emb = self.time_mlp(t) # (B, time_emb_dim)` `# Encoder` `e1 = self.enc1(x)` `e1 = e1 + self.time_proj1(t_emb)[:, :, None, None] # 广播加到每个空间位置` `e2 = self.enc2(self.pool1(e1))` `e2 = e2 + self.time_proj2(t_emb)[:, :, None, None]` `# Bottleneck` `b = self.bottleneck(self.pool2(e2))` `b = b + self.time_proj_bn(t_emb)[:, :, None, None]` `# Decoder (concat skip connection, 和标准 UNet 一样)` `d2 = self.up2(b)` `d2 = self.dec2(torch.cat([d2, e2], dim=1)) # skip connection` `d2 = d2 + self.time_proj3(t_emb)[:, :, None, None]` `d1 = self.up1(d2)` `d1 = self.dec1(torch.cat([d1, e1], dim=1)) # skip connection` `d1 = d1 + self.time_proj4(t_emb)[:, :, None, None]` `return self.final(d1)``# =====``# Part 3: GaussianDiffusion — DDPM 的核心算法``# =====``class GaussianDiffusion:` `"""` `实现 DDPM 的三个核心操作:` `1. 前向加噪  $q(x_t | x_0)$  — 2.2.2 节闭式解` `2. 训练损失  $L_{\text{simple}}$  — 2.4.4 节简化目标` `3. 反向采样  $p_\theta(x_{t-1} | x_t)$  — 2.6.1 DDPM / 2.6.2 DDIM` `"""``def __init__(self, T=1000, beta_start=1e-4, beta_end=0.02, device='cpu'):` `self.T = T` `self.device = device` `# -----  $\beta$  schedule (2.2 节) -----` `# 线性 schedule:  $\beta$  从  $1e-4$  线性增长到  $0.02$`  `# 含义: 噪声强度随时间步递增, 早期加少量噪声, 后期加大量噪声` `self.betas = torch.linspace(beta_start, beta_end, T, device=device)` `# ----- 从  $\beta$  推导所有系数 (2.2.2 节) -----` `#  $\alpha_t = 1 - \beta_t$  每步保留信号的比例` `self.alphas = 1.0 - self.betas` `#  $\bar{\alpha}_t = \alpha_1 \cdot \alpha_2 \cdot \dots \cdot \alpha_t$  累积保留比例 (随  $t$  递减, 趋向 0)` `self.alpha_bars = torch.cumprod(self.alphas, dim=0)` `# 预计算: 避免采样时反复做  $\sqrt{\alpha}$`  `self.sqrt_alpha_bars = torch.sqrt(self.alpha_bars)`

已赞同 1



添加评论

★ 收藏

♥ 喜欢

➔ 分享

📄 申请转载



...

```

# 前向过程  $q(x_t | x_0)$  — 对应 2.2.2 节闭式解
# -----
def q_sample(self, x0, t, noise=None):
    """
    一步从  $x_0$  跳到任意  $x_t$ （不需要逐步加噪）。

    公式:  $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$ ,  $\epsilon \sim N(0, I)$ 
           ^^^^^^^^^      ^^^^^^^^^^^^^^^^^
           信号部分      噪声部分

     $\alpha_t$  随  $t$  递减:  $t$  小时信号占主导,  $t$  大时噪声占主导。
    """
    if noise is None:
        noise = torch.randn_like(x0)

    # view(-1,1,1,1) 使系数维度匹配 (B,C,H,W), 支持广播
    sqrt_alpha_bar = self.sqrt_alpha_bar[t].view(-1, 1, 1, 1)
    sqrt_one_minus = self.sqrt_one_minus_alpha_bar[t].view(-1, 1, 1, 1)

    return sqrt_alpha_bar * x0 + sqrt_one_minus * noise

# -----
# 训练损失  $L_{\text{simple}}$  — 对应 2.4.4 节
# -----
def training_loss(self, model, x0):
    """
    DDPM 的简化损失函数: 随机选一个时间步  $t$ , 加噪后让网络预测噪声。

    算法流程:
    1. 均匀采样  $t \sim \text{Uniform}\{0, 1, \dots, T-1\}$ 
    2. 采样噪声  $\epsilon \sim N(0, I)$ 
    3. 用闭式解计算  $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$ 
    4. 网络预测  $\epsilon_\theta = \text{model}(x_t, t)$ 
    5. 损失 =  $\|\epsilon - \epsilon_\theta\|^2$ 

    为什么这个损失能工作? 因为  $L_2$  损失的最优解是条件期望 (2.7.2 节),
    所以网络会学到  $\epsilon_\theta^*(x_t, t) = E[\epsilon | x_t]$ , 即给定加噪图像后噪声的均值。
    """
    batch_size = x0.shape[0]

    # 1. 每个样本随机选一个时间步
    t = torch.randint(0, self.T, (batch_size,), device=self.device)

    # 2. 采样标准高斯噪声
    noise = torch.randn_like(x0)

    # 3. 前向加噪
    x_t = self.q_sample(x0, t, noise)

    # 4. 网络预测噪声
    predicted_noise = model(x_t, t)

    # 5. MSE 损失
    return F.mse_loss(predicted_noise, noise)

# -----
# DDPM 采样 — 对应 2.6.1 节
#

```

已赞同 1



添加评论

★ 收藏

♥ 喜欢

➔ 分享

📄 申请转载



已赞同 1   添加评论  收藏  喜欢  分享  申请转载  ...

```

x = torch.randn(shape, device=self.device)

for i, t in enumerate(timesteps):
    t_batch = torch.full((shape[0],), t, dtype=torch.long, device=self.de
    predicted_noise = model(x, t_batch)

    alpha_bar_t = self.alpha_bars[t]

    # Step 1: 预测  $\hat{x}_0$ 
    x0_pred = (x - torch.sqrt(1 - alpha_bar_t) * predicted_noise) / torch
    # clip 防止数值爆炸（高噪声时  $\bar{\alpha}_t \approx 0$ , 除法不稳定）
    x0_pred = torch.clamp(x0_pred, -1.0, 1.0)

    # 确定下一个时间步的  $\bar{\alpha}$ 
    if i < len(timesteps) - 1:
        alpha_bar_prev = self.alpha_bars[timesteps[i + 1]]
    else:
        alpha_bar_prev = torch.tensor(1.0, device=self.device) # t=0 时

    # Step 2: 确定性更新到  $x_{t-1}$ 
    x = torch.sqrt(alpha_bar_prev) * x0_pred + \
        torch.sqrt(1 - alpha_bar_prev) * predicted_noise

return x

```

```

# =====
# Part 4: 训练循环与使用示例
# =====

```

```
def train_diffusion(model, diffusion, dataloader, optimizer, epochs=100):
```

```
    """
```

```
    标准训练循环。对于 MNIST (28×28) 通常训练 50-100 个 epoch 即可看到效果。
```

```
    """
```

```
    model.train()
```

```
    for epoch in range(epochs):
```

```
        total_loss = 0
```

```
        for x_batch, _ in dataloader:
```

```
            x_batch = x_batch.to(diffusion.device)
```

```
            loss = diffusion.training_loss(model, x_batch)
```

```
            optimizer.zero_grad()
```

```
            loss.backward()
```

```
            optimizer.step()
```

```
            total_loss += loss.item()
```

```
    avg_loss = total_loss / len(dataloader)
```

```
    if (epoch + 1) % 10 == 0:
```

```
        print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")
```

```
# ===== 完整使用示例（以 MNIST 为例） =====
```

```
if __name__ == "__main__":
```

```
    from torchvision import datasets, transforms
```

```
    from torch.utils.data import DataLoader
```

```
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

已赞同 1



添加评论



收藏



喜欢



分享



申请转载



```

    ])
    dataset = datasets.MNIST(root="./data", train=True, download=True, transform=
    dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

# 2. 模型与扩散过程
model = SimpleUNet(in_channels=1, base_channels=64).to(device)
diffusion = GaussianDiffusion(T=1000, device=device)
optimizer = torch.optim.Adam(model.parameters(), lr=2e-4)

# 3. 训练
train_diffusion(model, diffusion, dataloader, optimizer, epochs=50)

# 4. 采样
model.eval()
# DDPM 采样 (1000 步, 慢但质量好)
samples_ddpm = diffusion.ddpm_sample(model, shape=(16, 1, 28, 28))
# DDIM 采样 (50 步, 快 20 倍, 质量接近)
samples_ddim = diffusion.ddim_sample(model, shape=(16, 1, 28, 28), num_steps=

print(f"DDPM samples shape: {samples_ddpm.shape}") # (16, 1, 28, 28)
print(f"DDIM samples shape: {samples_ddim.shape}") # (16, 1, 28, 28)
```

代码与公式的对应关系：

| 代码                                                     | 对应的数学公式                                                                                             | 文档章节                 |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------|
| SinusoidalTimeEmbedding                                | $PE(t, 2i) = \sin(t / 10000^{(2i/d)})$                                                              | 与 Transformer 位置编码相同 |
| $\sqrt{\alpha\_bar} * x_0 + \sqrt{one\_minus} * noise$ | $x_t = \sqrt{(\alpha_t)} \cdot x_0 + \sqrt{(1 - \alpha_t)} \cdot \epsilon$                          | 2.2.2 闭式解            |
| F.mse_loss(predicted_noise, noise)                     | $L =   \epsilon - \epsilon_\theta(x_t, t)  ^2$                                                      | 2.4.4 简化损失           |
| DDPM 的 mean 计算                                         | $\mu_\theta = (1/\sqrt{\alpha_t})(x_t - \beta_t / \sqrt{(1 - \alpha_t)} \cdot \epsilon_\theta)$     | 2.6.1 DDPM 采样        |
| DDPM 的 mean + sigma * z                                | $x_{t-1} = \mu_\theta + \sigma_t z$ (反向 SDE)                                                        | 2.6.1 DDPM 采样        |
| DDIM 的 x0_pred                                         | $\hat{x}_0 = (x_t - \sqrt{(1 - \alpha_t)} \cdot \epsilon_\theta) / \sqrt{(\alpha_t)}$               | 2.6.2 DDIM Step 1    |
| DDIM 的确定性更新                                            | $x_{t-1} = \sqrt{(\alpha_{t-1})} \cdot \hat{x}_0 + \sqrt{(1 - \alpha_{t-1})} \cdot \epsilon_\theta$ | 2.6.2 DDIM Step 2    |
| time_proj(t_emb)[:,None,None] 加到特征图                    | 时间步条件注入：将 t 的编码加到每层特征上                                                                              | 条件生成的标准做法            |

本章小结

- 核心突破：把 VAE 的“一步跳”变成“多步走”——隐变量从 z 变成整条马尔可夫链  $x_0, x_1, ..., x_T$
- 前向过程闭式解： $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$ （一步算出任意时间步的噪声图）
- 反向过程推导链： $q(x_{t-1}|x_t)$  不可算 → 给定  $x_0$  后可算 → 用  $\epsilon$  替换  $x_0$  → 训练网络预测  $\epsilon$
- 预测噪声 = 学 score： $\epsilon_\theta \approx -\sqrt{1 - \alpha_t} \nabla_{x_t} \log q(x_t)$ ，连接了 Diffusion 和 Score Matching 两个框架
- 三种预测目标 ( $\epsilon, x_0, v$ ) 数学上等价，选哪个是工程和经验的权衡

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D直答

消息

私信

（一）Flow Matching 进阶版 付予这篇内容一个「理性与画 VS 拉皇」的安利，从进阶版开始，从线性方程出发，讲清为何边缘速度场算不出来、Conditional Flow Matching 如何与 Diffusion 的「给定  $x_0$  后再算」同构，以及线性插值、采样器与和 Diffusion 的统一图景。

送礼物

还没有人送礼物，鼓励一下作者吧

编辑于 2026-04-25 22:59 · 北京

Diffusion

## 爆款海外云主机 低至2.5折

凭借覆盖全球云服务节点，具有竞争力的云服务，多项国际权威认证和专业的合规建... [查看详情](#)

天翼云 的广告

X



理性发言，友善互动

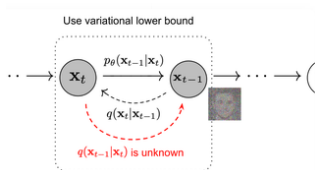
☐ 同时发布到想法

发布



还没有评论，发表第一个评论吧

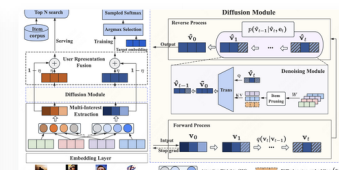
## 推荐阅读



### Diffusion Models - 扩散模型 (一)

siyoi'

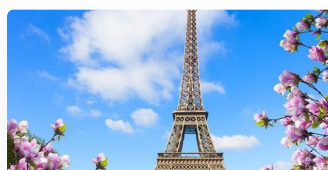
发表于 2026-04-25



### 「小红书」扩散模型做多兴趣匹配 | Diffusion Model for...

猫猫的梦中情猫

发表于 2026-04-25



### 扩散模型中的 Noise Scheduler

去去

发表于 2026-04-25

## Diffusion 模型关键知识

什么是score，为什么需要score？  
如果我们要直接建模一个概率密度，我们需要知道归一化系数 但是一般来说这很难求，所以需要用到 郎之万马尔可夫蒙特卡洛采样的方式，采样不一定要建模概率密...

消息 bal...

发表于 Diffu...

已赞同 1

添加评论

★ 收藏

♥ 喜欢

➔ 分享

📄 申请转载

✦

...